UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº04

NOMBRE COMPLETO: Valdes Trejo Rodrigo Alfredo

Nº de Cuenta: 318224754

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: jueves 12 de marzo de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Terminar la Grúa con:
-cuerpo(prisma rectangular)
-base (pirámide cuadrangular)
-4 llantas(4 cilindros) con teclado se pueden girar cada una de las 4 llantas por separado

Base Se recupera la matriz original de la cabina con modelaux2 y se traslada 1.0f hacia abajo en Y. Se renderiza usando una pirámide cuadrangular (meshList[4]) escalada a (6, 2.5, 4) de color verde azulado, representando la base estructural sobre la que descansa toda la grúa.

```
//base
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(6.0f, 2.5f, 4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 1.0f, 0.6f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[4]->RenderMesh(); //piramide cuadrangular
```

Figura 1
Fragmento de código de la base de la grúa formada con una pirámide cuadrangular.
Nota. Elaboración propia.

Ruedas Se implementan 4 ruedas usando cilindros (meshList[2]) de color dorado, todas heredando desde modelaux2 de forma independiente entre sí. Cada rueda sigue el mismo patrón: se traslada a su esquina correspondiente de la base, se le aplica una rotación en Z controlada por su tecla asignada para simular el giro, y finalmente se rota 90° en X para orientar el cilindro horizontalmente como una rueda. Sus posiciones y teclas son: K, L, N, M.

Vale la pena notar que cada rueda tiene su **propia articulación independiente**, lo que significa que pueden girar de forma individual sin afectar a las demás ni a ningún otro elemento de la jerarquía, ya que todas parten directamente de modelaux2 en lugar de heredar de la cadena del brazo.

```
//r1
model = modelaux2;
model = glm::translate(model, glm::vec3(-3.0f, -2.25f, -2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, -1.0f));//tecla k
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.9f, 0.7f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); //cilindro
//R2
model = modelaux2;
model = glm::translate(model, glm::vec3(-3.0f, -2.25f, 2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, -1.0f));//tecla l
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.9f, 0.7f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
//R3
model = modelaux2;
model = glm::translate(model, glm::vec3(3.0f, -2.25f, -2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, -1.0f));//tecla n
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.9f, 0.7f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
//R4
model = modelaux2;
model = glm::translate(model, glm::vec3(3.0f, -2.25f, 2.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, -1.0f));//tecla m
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
```

Figura 2
Fragmento de código de las ruedas de la grúa con rotaciones independientes.
Nota. Elaboración propia.

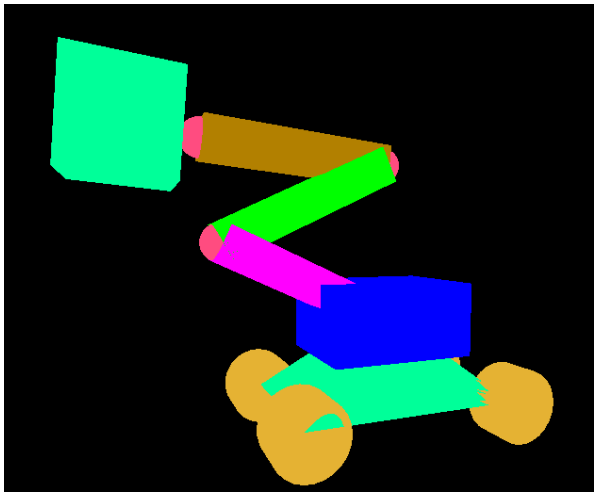

Figura 3
Captura de pantalla de la grúa completa.
Nota. Elaboración propia.

2.- Crear un animal robot 3d

-Instanciando cubos, pirámides, cilindros, conos, esferas:

-4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)

-cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente

Cuerpo Es la base del caballo. Se posiciona en (0, 3, -4) y se guarda en modelaux y modelaux3 para que todos los demás elementos hereden su posición. Se escala a (10, 3, 5) de color rosa, representando el torso principal del animal.

```
//cuerpo
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 3.0f, -4.0f));    //cabina 0,3,0
modelaux = model;//lo de arriba se hereda
modelaux3 = model;
model = glm::scale(model, glm::vec3(10.0f, 3.0f, 5.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.5f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo
```

Figura 4

Fragmento de código del cuerpo del animal modelado.

Nota. Elaboración propia.

Cuello Hereda desde modelaux del cuerpo. Se aplica primero la rotación de la articulación 1 controlada por tecla F, luego se rota 135° en Z y se traslada para posicionarlo en la parte delantera superior del cuerpo. Se escala como un cubo alargado (5, 2, 2) de color magenta.

```
//cuello
model = modelaux;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));//tecla F
model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(6.0f, 2.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
```

Figura 5

Fragmento de código del cuello del animal modelado con la asignación de tecla para moverlo.

Nota. Elaboración propia.

Cabeza Hereda del cuello. Se rota 90° en Z y se traslada para posicionarla al final del cuello. Su matriz se guarda en modelaux2 para que las orejas la hereden. Se escala a (3, 2, 2) de color naranja.

```
model = modelaux;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(2.5f, -1.5f, 0.0f));
modelaux = model;
modelaux2 = model;
model = glm::scale(model, glm::vec3(3.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix())
color = glm::vec3(1.0f, 0.5f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
```

Figura 6
Fragmento de código de la cabeza del animal modelado.
Nota. Elaboración propia.

Orejas Se crean dos orejas simétricas heredando desde modelaux2 de la cabeza. Ambas usan una pirámide cuadrangular (meshList[4]) escalada a (1, 1.5, 1) de color azul, separadas en Z con valores 0.55f y -0.55f para posicionarlas a cada lado de la cabeza.

```
//orejas
model = modelaux2;
model = glm::translate(model, glm::vec3(-2.5f, -1.5f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.5f, 1.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.5f, 0.15f, 0.55f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.3f, 0.1f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[4]->RenderMesh(); //dibuja cubo y pirámide triangular
//orejas
model = modelaux2;
model = glm::translate(model, glm::vec3(-2.5f, -1.5f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.5f, 1.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.5f, 0.15f, -0.55f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.3f, 0.1f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[4]->RenderMesh(); //dibuja cubo y pirámide triangular
```

Figura 7
Fragmento de código de las orejas del animal modelado, no tienen movimiento.
Nota. Elaboración propia.

Pata trasera derecha Hereda desde modelaux3 del cuerpo directamente. Se posiciona en (4.5, -0.5, 3.25) y tiene una rotación en Z controlada por tecla G (getarticulacion2()). Se compone de dos segmentos de cubo morado separados por una articulación esfera rosa controlada por tecla H (getarticulacion3()), simulando el muslo y la pantorrilla de la pata.

```
model = modelaux3;
model = glm::translate(model, glm::vec3(4.5f, -0.5f, 3.25f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, -1.0f));//tecla g
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.9f, 0.7f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular

model = modelaux;
model = glm::translate(model, glm::vec3(0.0, -2.75f, 0.0f));//4.5f, -2.5f, 2.5f
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 3.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.5f, 0.0f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular

//articulacion
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));//tecla H
modelaux = model;
model = glm::scale(model, glm::vec3(0.75f, 0.75f, 0.75f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.3f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0, -2.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 3.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
```

Figura 8

Fragmento de código de la pata trasera del animal modelado que tiene movimiento mediante las teclas G y H.

Nota. Elaboración propia.

Pata trasera izquierda Igual estructura que la pata trasera derecha pero posicionada en (4.5, -0.5, -3.25). Su articulación superior es controlada por tecla J (getarticulacion4()) y la articulación inferior por tecla K (getarticulacion5()).

```

model = modelaux3;
model = glm::translate(model, glm::vec3(4.5f, -0.5f, -3.25f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, -1.0f));//tecla j
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.9f, 0.7f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular

model = modelaux;
model = glm::translate(model, glm::vec3(0.0, -2.75f, 0.0f));//4.5f, -2.5f, 2.5f
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 3.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.5f, 0.0f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular

//articulacion
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));//tecla k
modelaux = model;
model = glm::scale(model, glm::vec3(0.75f, 0.75f, 0.75f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.3f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0, -2.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 3.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
```

Figura 9

Fragmento de código de la pata trasera del animal modelado que tiene movimiento mediante las teclas J y K.

Nota. Elaboración propia.

Pata delantera derecha Hereda desde modelaux3 del cuerpo. Se posiciona en (-4.5, -1.0, 3.0) en el lado delantero del caballo. Usa cilindros más pequeños con escala 0.8 en comparación con las patas traseras. Su articulación superior es controlada por tecla L (getarticulacion6()) y la articulación inferior por tecla N (getarticulacion7()).

```
model = modelaux3;
model = glm::translate(model, glm::vec3(-4.5f, -1.0f, 3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, -1.0f));//tecla L
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.9f, 0.7f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular

model = modelaux;
model = glm::translate(model, glm::vec3(0.0, -2.25f, 0.0f));//4.5f, -2.5f, 2.5f
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 3.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformVie  (variable local) GLuint uniformProjection  ıera.calculateViewMatrix()));
color = glm::vec3(0.5f, 0.0f,
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular

//articulacion
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));//tecla N
modelaux = model;
model = glm::scale(model, glm::vec3(0.75f, 0.75f, 0.75f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.3f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0, -2.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 3.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
```

Figura 10

Fragmento de código de la pata trasera del animal modelado que tiene movimiento mediante las teclas G y H.

Nota. Elaboración propia.

Pata delantera izquierda Misma estructura que la pata delantera derecha pero posicionada en (-4.5, -1.0, -3.0). Su articulación superior es controlada por tecla M (getarticulacion8()) y la articulación inferior por tecla B (getarticulacion9()).

```
//pata delantera
model = modelaux3;
model = glm::translate(model, glm::vec3(-4.5f, -1.0f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, -1.0f));//tecla M
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.9f, 0.7f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); //dibuja cubo y pirámide triangular

model = modelaux;
model = glm::translate(model, glm::vec3(0.0, -2.25f, 0.0f));//4.5f, -2.5f, 2.5f
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 3.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.5f, 0.0f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular

//articulacion
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));//tecla B
modelaux = model;
model = glm::scale(model, glm::vec3(0.75f, 0.75f, 0.75f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.3f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();

model = modelaux;
model = glm::translate(model, glm::vec3(0.0, -2.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.5f, 3.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.5f, 0.0f, 0.5f);
```

Figura 11

Fragmento de código de la pata trasera del animal modelado que tiene movimiento mediante las teclas M y B.

Nota. Elaboración propia.

Cola Hereda desde modelaux3 del cuerpo. Se aplica primero una rotación en X controlada por tecla 10 (getarticulacion10()) para el movimiento de la cola, luego se rota 120° en Z y se traslada a la parte trasera del cuerpo. Se escala como un cubo alargado (5, 1, 1) de color magenta, representando la cola del caballo.

```
//cola
model = modelaux3;
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(1.0f, 0.0f, 0.0f));//tecla V
model = glm::rotate(model, glm::radians(120.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(-4.5f, -4.5f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
```

Figura 12

Fragmento de código de la cola del animal modelado que tiene movimiento mediante las tecla V.

Nota. Elaboración propia.

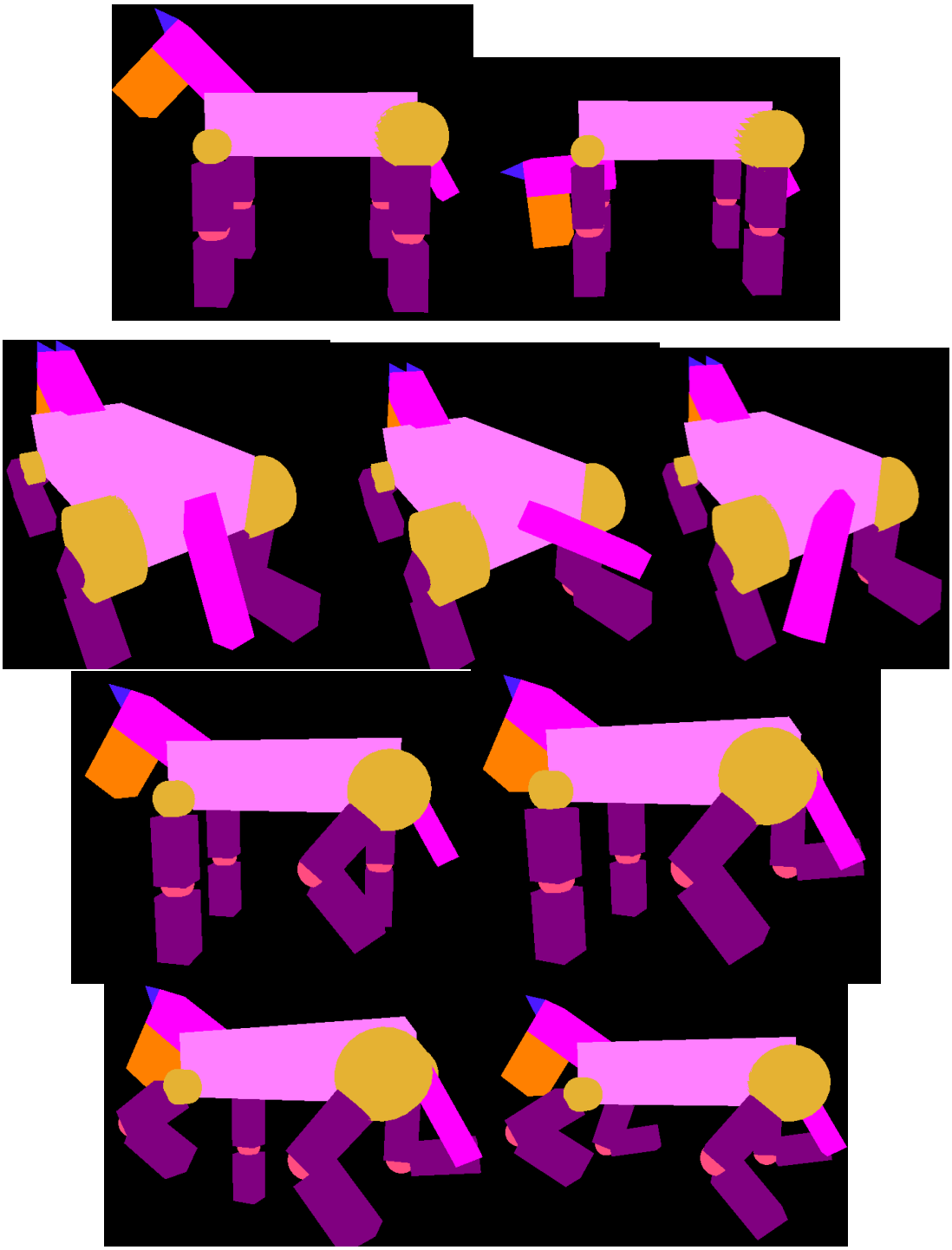

Figura 13
Capturas de pantalla del anima modelado con los distintos movimientos
Nota. Elaboración propia.

Modificaciones archivo Window.h y Window.cpp

Para poder articular ambos modelos hicieron falta asignar teclas adicionales por lo tanto de se agregaron las articulaciones: 7,8,9 y 10. Las modificaciones en los archivos fueron las siguientes:

Window.cpp

```cpp
{
    width = windowWidth;
    height = windowHeight;
    rotax = 0.0f;
    rotay = 0.0f;
    rotaz = 0.0f;
    articulacion1 = 0.0f;
    articulacion2 = 0.0f;
    articulacion3 = 0.0f;
    articulacion4 = 0.0f;
    articulacion5 = 0.0f;
    articulacion6 = 0.0f;
    articulacion7 = 0.0f;
    articulacion8 = 0.0f;
    articulacion9 = 0.0f;
    articulacion10 = 0.0f;

    for (size_t i = 0; i < 1024; i++)
    {
        keys[i] = 0;
```

```cpp
    {
        theWindow->articulacion4 += 10.0;
    }
    if (key == GLFW_KEY_K)
    {
        theWindow->articulacion5 += 10.0;
    }
    if (key == GLFW_KEY_L)
    {
        theWindow->articulacion6 += 10.0;
    }
    if (key == GLFW_KEY_N)
    {
        theWindow->articulacion7 += 10.0;
    }
    if (key == GLFW_KEY_M)
    {
        theWindow->articulacion8 += 10.0;
    }
    if (key == GLFW_KEY_B)
    {
        theWindow->articulacion9 += 10.0;
    }
    if (key == GLFW_KEY_V)
    {
        theWindow->articulacion10 += 10.0;
    }
```

Window.h

```cpp
    GLfloat getarticulacion6() { return articulacion6; }
    GLfloat getarticulacion7() { return articulacion7; }
    GLfloat getarticulacion8() { return articulacion8; }
    GLfloat getarticulacion9() { return articulacion9; }
    GLfloat getarticulacion10() { return articulacion10; }

    ~Window();
ivate:
    GLFWwindow *mainWindow;
    GLint width, height;
    GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6, articulac
    bool keys[1024];
```

3.- Conclusión

En esta práctica se reforzó el concepto de modelado jerárquico mediante la implementación de dos figuras articuladas: una grúa y un caballo, cada una con sus propias particularidades estructurales que permitieron profundizar en el uso de matrices auxiliares y cadenas de transformaciones en OpenGL.

El ejercicio del caballo resultó especialmente ilustrativo ya que, a diferencia de la grúa, cuya jerarquía es lineal y sigue una sola cadena de elementos, el caballo requirió manejar múltiples cadenas jerárquicas independientes que parten de un mismo origen. Las cuatro patas, la cola, el cuello y la cabeza heredan todas desde la matriz del cuerpo principal almacenada en modelaux3, pero cada una sigue su propia rama jerárquica de forma independiente, lo que refleja con mayor fidelidad la estructura real de un ser vivo.

Además, la implementación de patas compuestas por dos segmentos separados por una articulación esfera permitió simular el comportamiento de una extremidad con rodilla, donde cada segmento responde a su propia articulación controlada por teclado sin afectar a las demás patas, demostrando la flexibilidad y potencia del modelado jerárquico para representar estructuras biológicas complejas.

En conjunto, ambos ejercicios consolidaron la comprensión de cómo el orden y la correcta gestión de las matrices de transformación son fundamentales para construir modelos articulados coherentes, sentando bases sólidas para el desarrollo de animaciones y simulaciones de personajes o mecanismos en proyectos futuros de computación gráfica.

4.- Bibliografia

- Khronos Group. (2024). *OpenGL 4.6 API core profile specification*. Recuperado el 9 de marzo de 2026, de https://registry.khronos.org/OpenGL/specs/gl/glspec46.core.pdf
- Khronos Group. (2024). *GLSL the OpenGL shading language specification version 4.60*. Recuperado el 9 de marzo de 2026, de https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf
- De Vries, J. (2024). *Transformations*. LearnOpenGL. Recuperado el 9 de marzo de 2026, de https://learnopengl.com/Getting-started/Transformations